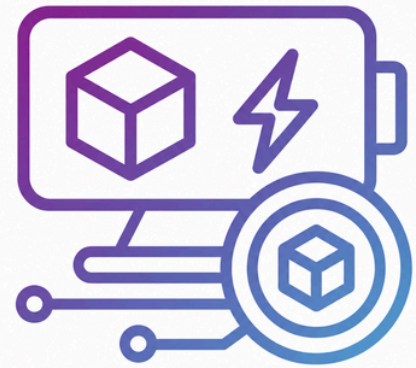




Python 미니 프로젝트

EV 배터리 진단 및 정비소·충전소 연계 플랫폼

<https://github.com/JIWOO1127/python-mini-team-project>

팀명 2조

팀원 서지우, 윤은빈, 지태현, 이태인, 홍인기, 김유석

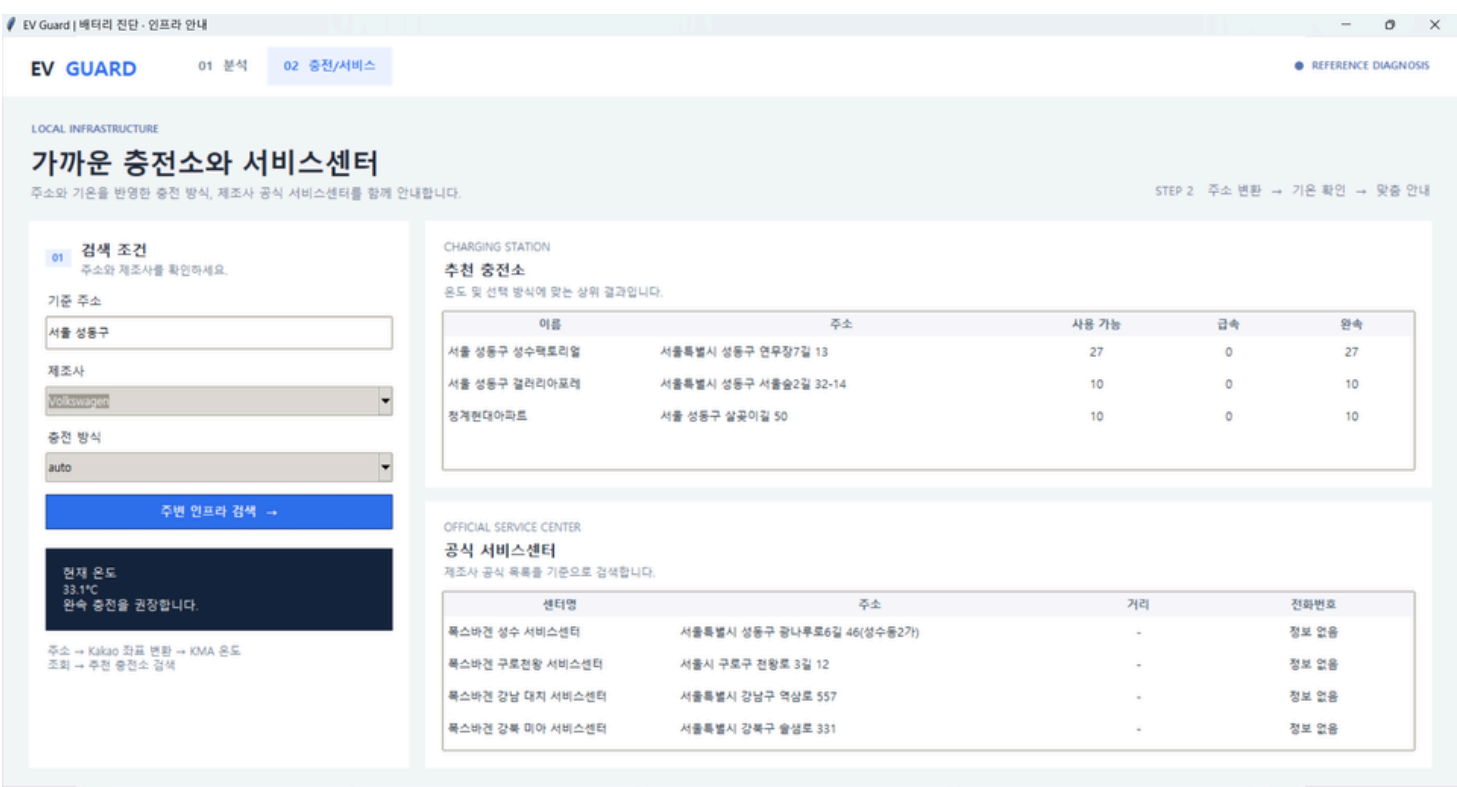
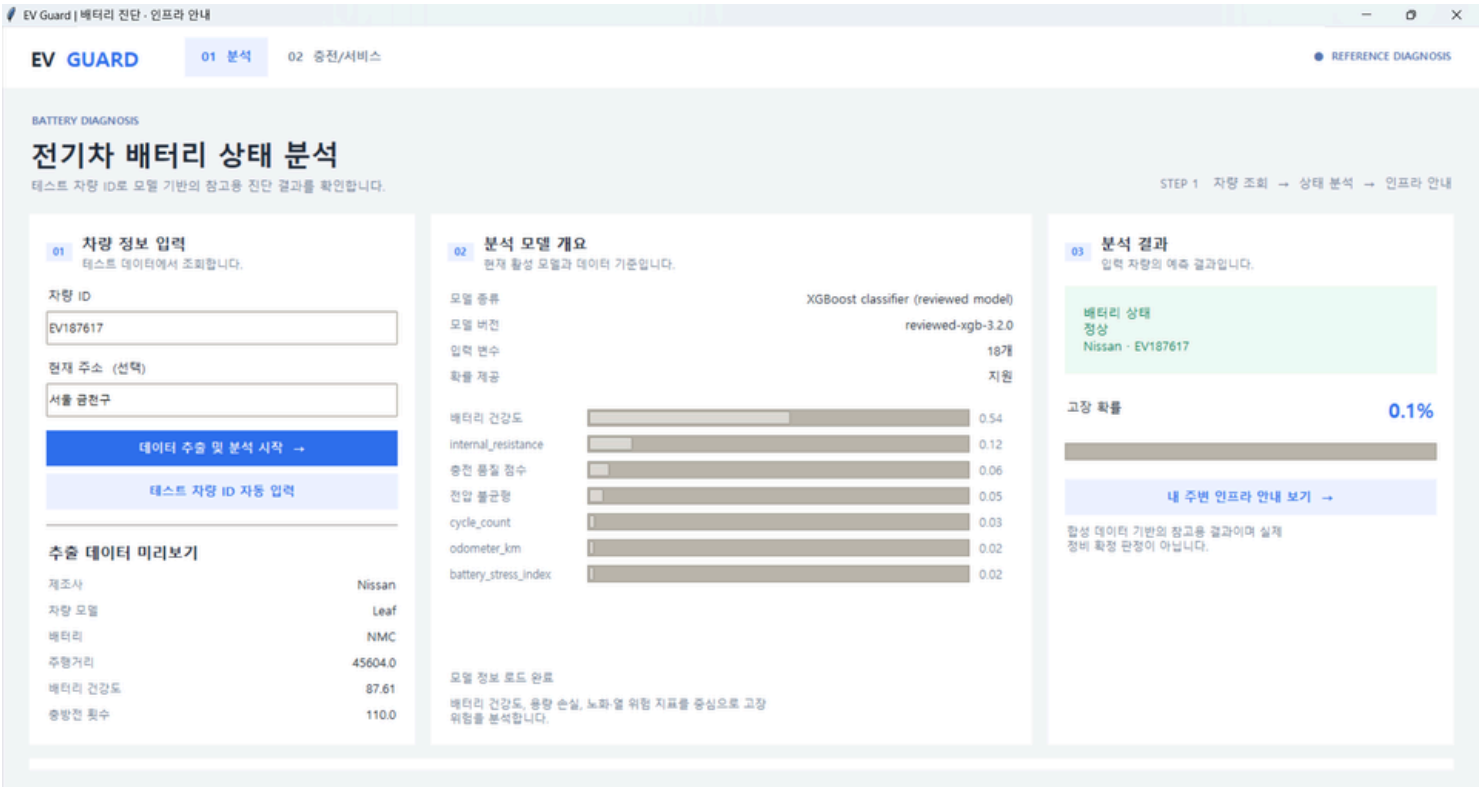
목차

0. 프로젝트 개요
1. 데이터 수집 및 분석
2. 모델링 및 평가
3. 크롤링 및 API
4. 시스템 설계 및 구현
5. 결론 및 향후 개선

프로젝트 개요

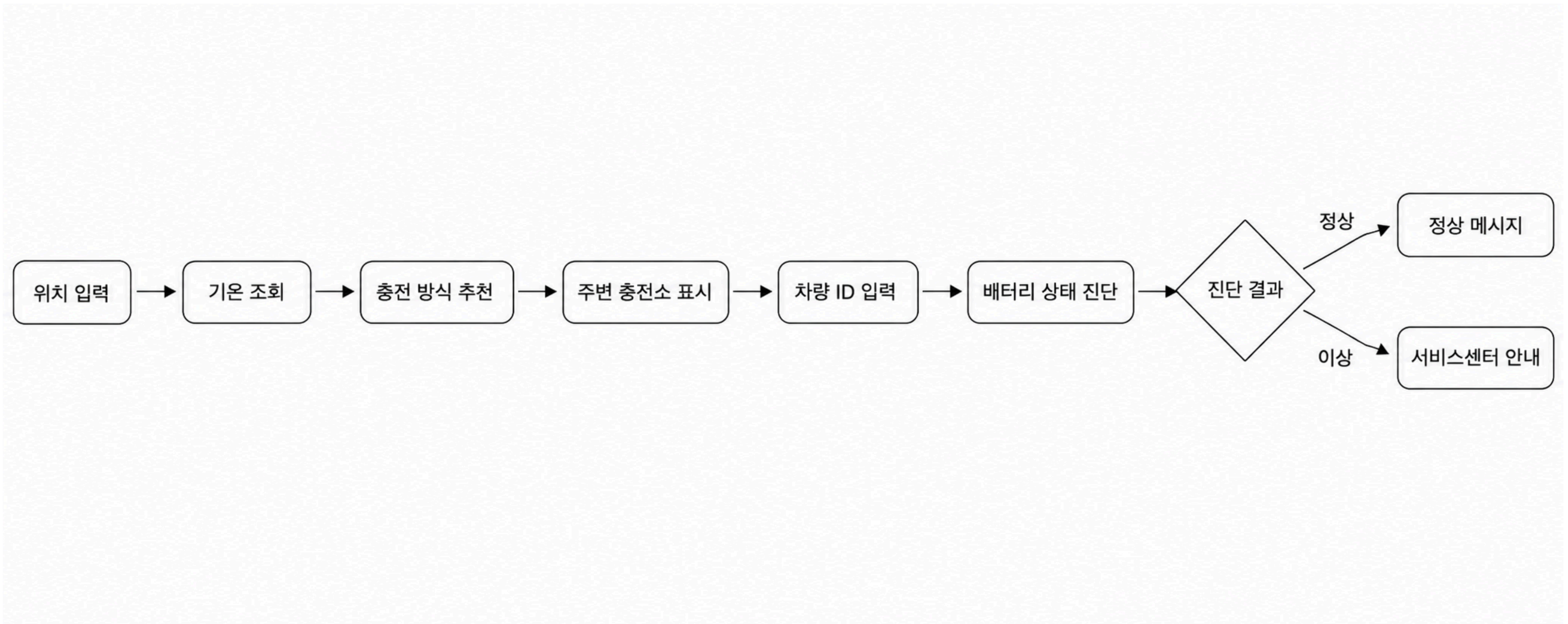
✳ 프로젝트 배경 및 목적

- 전기차 보급 확대로 배터리 고장·열화에 대한 사전 관리 수요 증가
- 그러나 일반 사용자가 배터리 상태를 직접 판단하기 어려움
- EV Guard: 차량 ID 하나로 배터리 상태 진단부터 주변 충전소·서비스센터 안내까지 원스톱으로 제공하는 서비스
- UI 01 분석: 센서 데이터 기반 배터리 불량 예측
- UI 02 충전/서비스: 기온 기반 충전 방식 추천 + 가까운 충전소·서비스센터 안내



프로젝트 개념도

✳ 사용자 시나리오

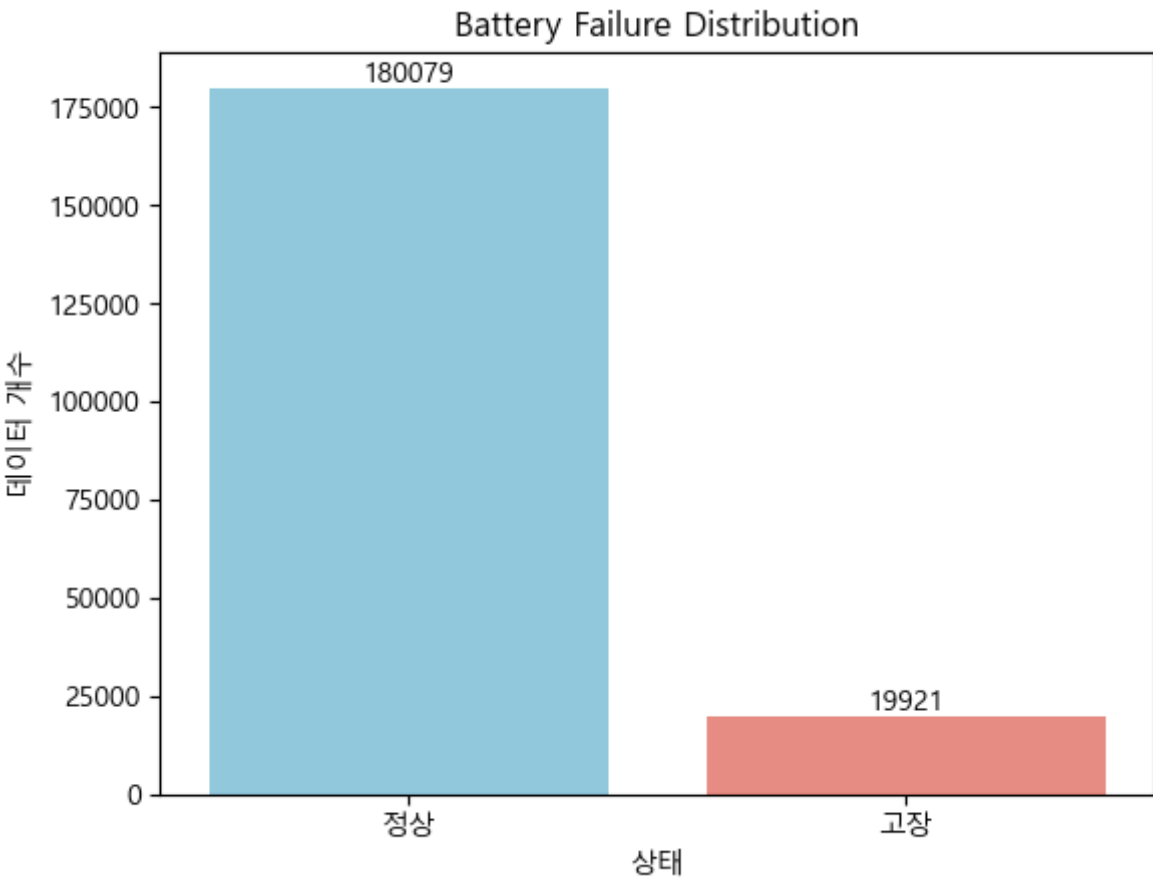


프로젝트 개요

❄ 데이터셋 소개 (EV Battery Failure Prediction Dataset)

- 샘플(레코드) 수: 200,000개 / 피쳐(특성) 수: 70개 / 불량률 (불균형) : 약 10%

항목	내용
타겟	battery_failure (0=정상, 1=불량)
브랜드	20개 (Tesla, BYD, Hyundai, BMW 등)
배터리 화학종	5종 (LFP, NMC, NCA, LMO, LTO)
데이터 특성	3~5% 결측, 0.6% 센서 이상치 포함
생성 방식	배터리 열화 물리·화학 기반 시뮬레이션



출처: Kaggle "EV Battery Failure Prediction Dataset (200K)"

<https://www.kaggle.com/datasets/sarveshchhetri/ev-battery-failure-prediction-dataset-200k>

프로젝트 개요

✱ 팀 구성 및 역할

훈련생	역할	담당업무	
홍인기	팀장	데이터 분석 및 모델링	결과 보고서 제작 및 검토
윤은빈	팀원	데이터 분석 및 모델링	결과 보고서 제작 및 검토
이태인	팀원	데이터 분석 및 모델링	결과 보고서 제작 및 검토
김유석	팀원	크롤링 및 UI	결과 보고서 제작 및 검토
서지우	팀원	크롤링 및 UI	결과 보고서 제작 및 검토
지태현	팀원	크롤링 및 UI	결과 보고서 제작 및 검토

프로젝트 개요

✳ 프로젝트 일정 및 절차

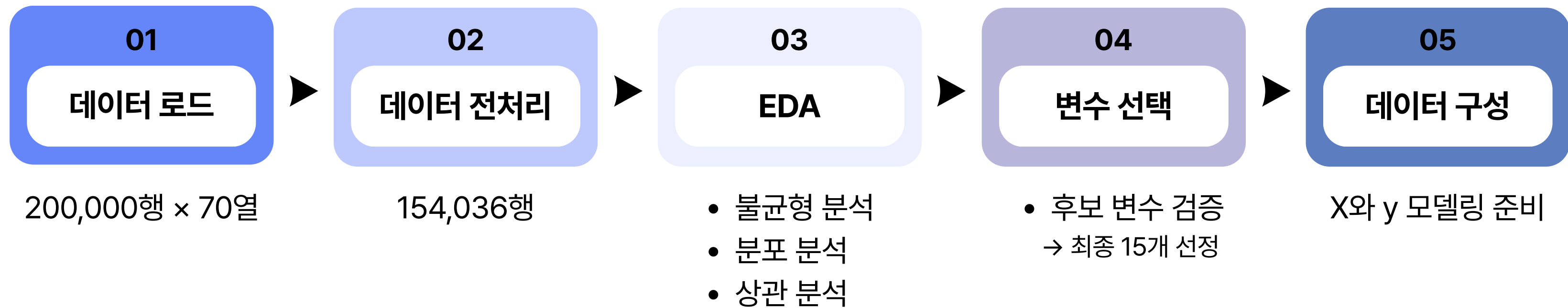
구분	기간	활동
프로젝트 설정 및 데이터 수집	8/25 15:00~18:00	주제 선정, Kaggle 데이터셋 탐색, 역할 분담, GitHub 세팅
데이터 전처리 및 UI구성	8/26 14:00~18:00	결측 행 제거, 브랜드 필터링, EDA(상관·분포·불균형 분석), UI 레이아웃 설계
모델 학습, UI설계	8/27 10:00 ~ 18:00	변수 선택, LightGBM/XGBoost 학습, 임계값 조정, 백엔드·UI 연동
보고서작성	8/28 09:00 ~ 15:00	모델 성능 평가, 최종 모델 저장(.pkl), 발표자료 및 보고서 작성

기술 스택 및 사용 라이브러리

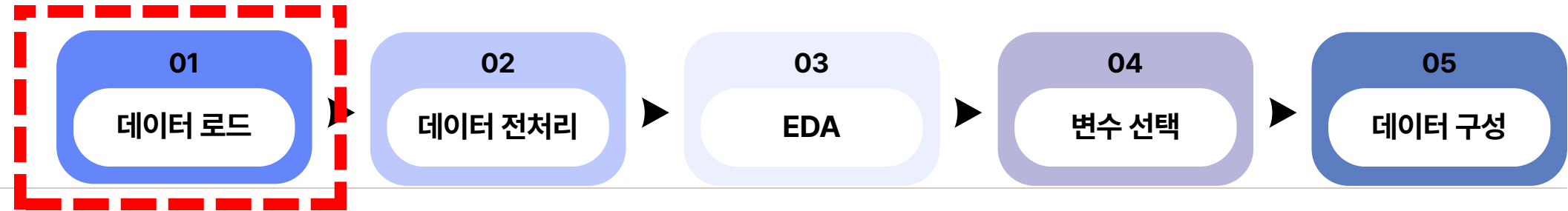
레이어	주요 기술	역할
데스크톱 UI	tkinter · threading · requests	2-Step 카드 UI, 비동기 워커로 네트워크 대기 중 화면 멈춤 방지
API 서버	FastAPI · Uvicorn · Pydantic v2	REST 엔드포인트, 요청·응답 스키마 검증, Provider 인터페이스로 외부 API 변경 격리
ML 엔진	LightGBM 4.7.0 · scikit-learn 1.7.2 · joblib	배터리 고장 이진 분류(피쳐 15종, 임계값 0.2), Imputer 결측 보정, 모델 번들 직렬화
데이터 처리	pandas	차량 CSV 조회, 모델 입력 구성, 검색 결과 CSV 캐시
외부 수집	BeautifulSoup4 · Selenium	충전소 HTML 파싱, 제조사 공식 카탈로그 수집(Headless Chrome)
외부 API	Kakao Local · 기상청 초단기실황	주소 → 위경도 지오코딩, 실시간 기온 기반 충전 방식 추천

데이터 수집 및 분석

* 데이터 분석 흐름



데이터 수집 및 분석



* 데이터 불러오기 및 구조 확인

• 데이터 규모

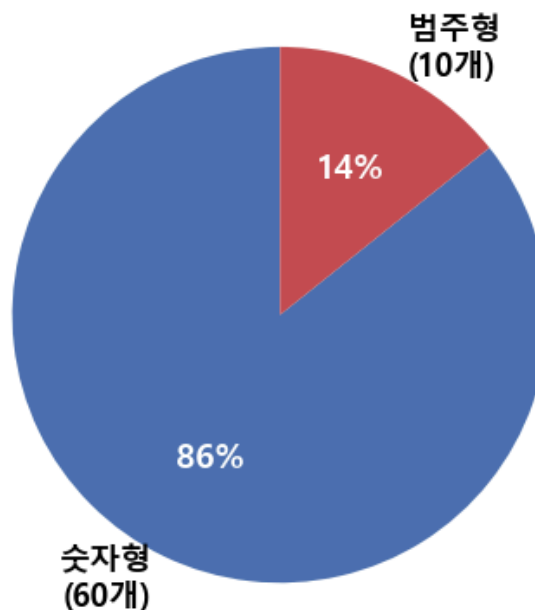
- 전체 샘플: **200,000개**
- 전체 변수: **70개**
- Target: **battery_failure**
- **정상(0) / 불량(1)**으로 구성

• 초기 데이터 확인 결과

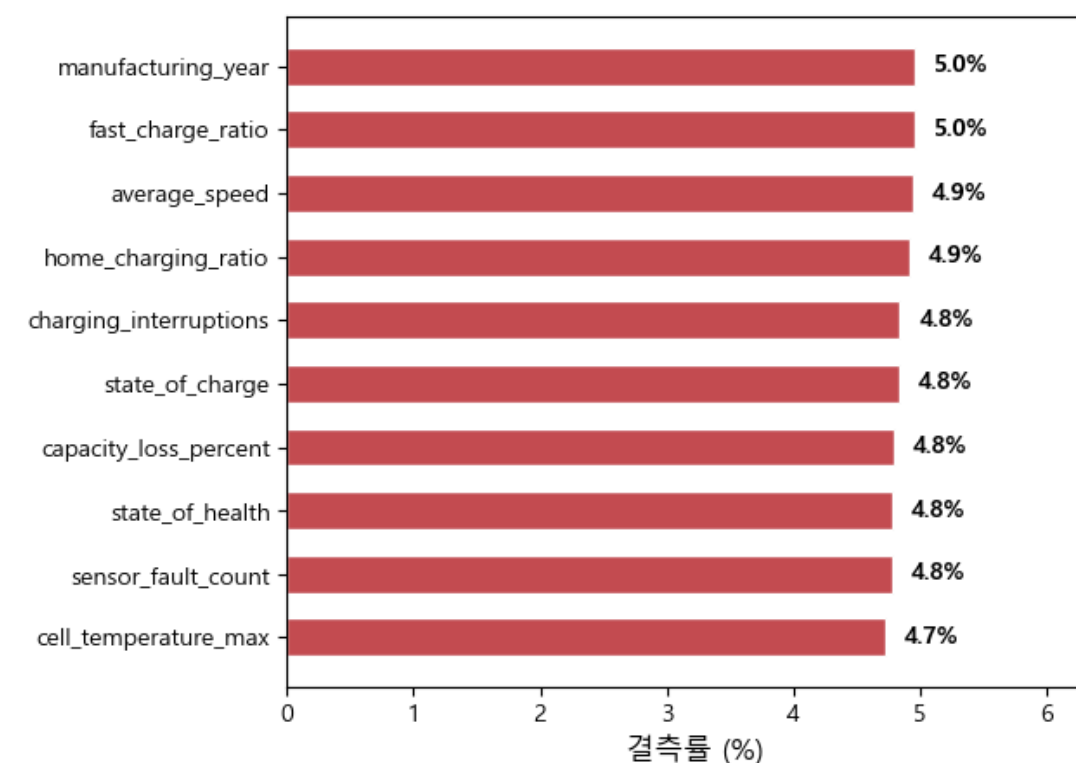
* 타겟·식별자 등 분석 대상에서 제외할 컬럼은 변수 선택 단계에서 분리

- 변수별 데이터 타입 확인 → **수치형 62개 / 범주형 8개**
- 결측치 및 이상 데이터 확인 → 67개 컬럼에 3~5% 결측 산재 → EDA ①에서 상세 분석
- 타겟 클래스 분포 확인 → 정상 90% / 불량 10% → EDA ②에서 상세 분석

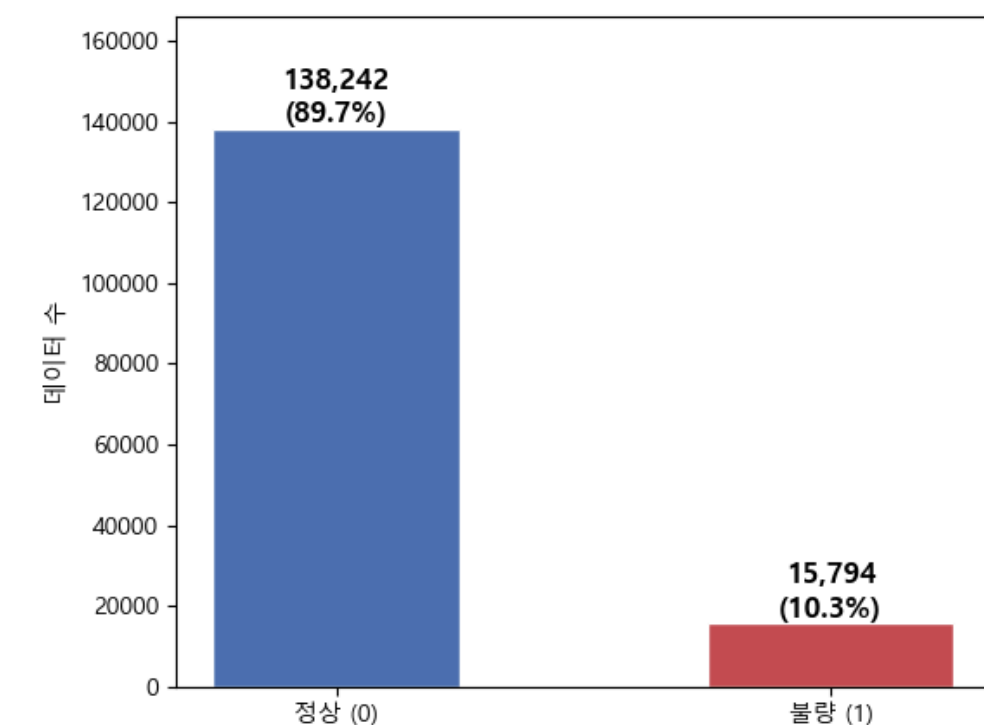
변수 데이터 타입 분포



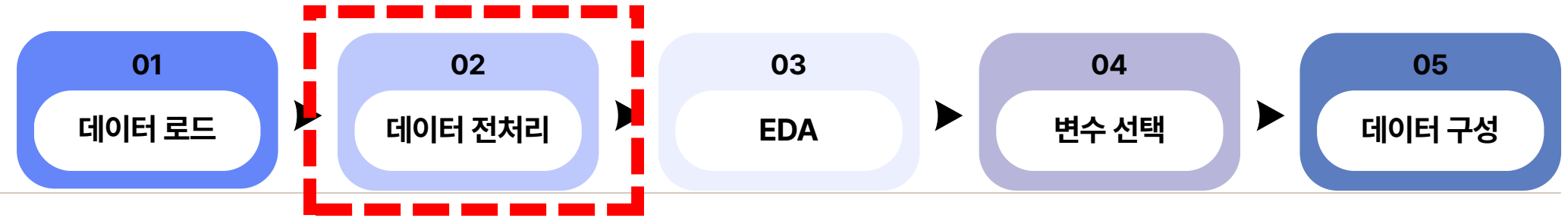
컬럼별 결측률 Top 10



타겟 클래스 분포



데이터 수집 및 분석



✳ 데이터 전처리

• 주요 범주형 변수 결측 행 제거

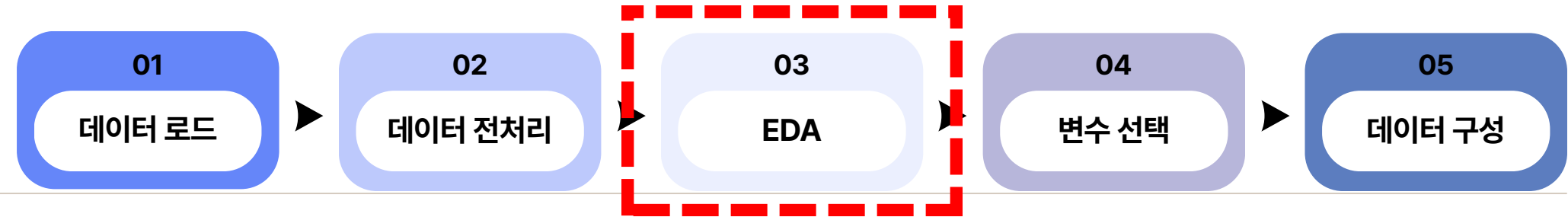
- 처리 대상 : **vehicle_brand, vehicle_model, vehicle_type**
- 처리 방법 : 3개 변수 중 하나라도 결측값(NaN)이 존재하는 행을 제거
- 처리 이유 : 차량 브랜드·모델·유형은 차량을 식별하고 서비스 대상을 구분하는 핵심 범주형 정보이므로, 해당 정보가 없는 데이터는 분석 대상에서 제외

• 국내 서비스 연계 대상 브랜드 필터링

- 처리 대상 : **차량 브랜드(vehicle_brand)**
- 처리 방법 : 국내 서비스센터 미보유 브랜드 5개 제외 (**Rivian · Lucid · Tata · Mahindra · MG**)
- 처리 이유 : 프로젝트가 배터리 이상 진단 이후 서비스센터 연계까지 고려한 시스템이므로, 국내 서비스 연계가 어려운 브랜드를 분석 대상에서 제외

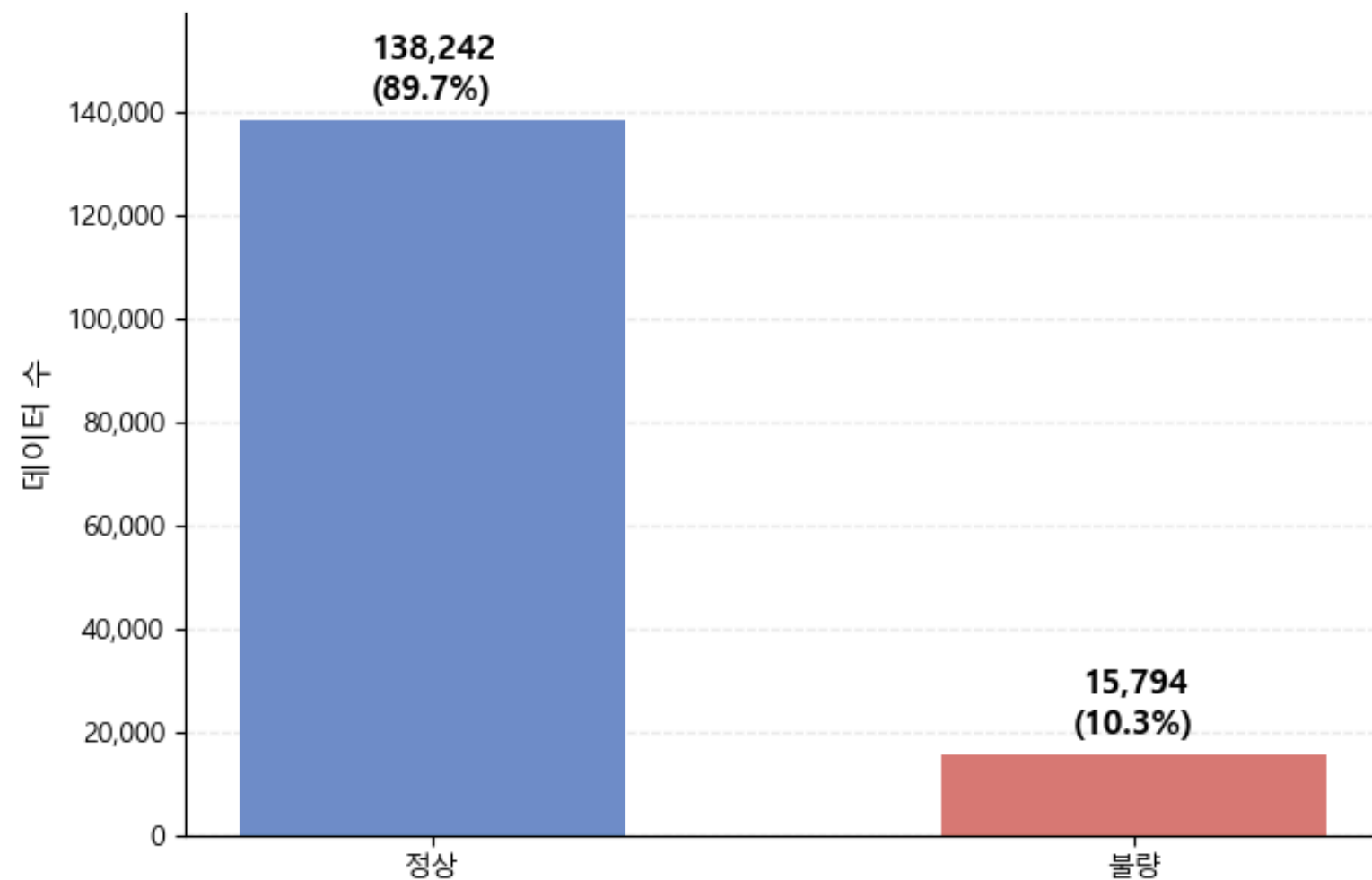
```
[결측 제거] 대상 20,408행  
→ 제거 후 179,592행  
[브랜드 제거] ['Rivian', 'Lucid', 'Tata', 'Mahindra', 'MG']  
→ 179,592행 → 154,036행 (25,556행 제거)
```

데이터 수집 및 분석



✳ EDA ② 타깃 클래스 분포 (불균형)

배터리 정상 · 불량 데이터 분포



• 타깃 변수

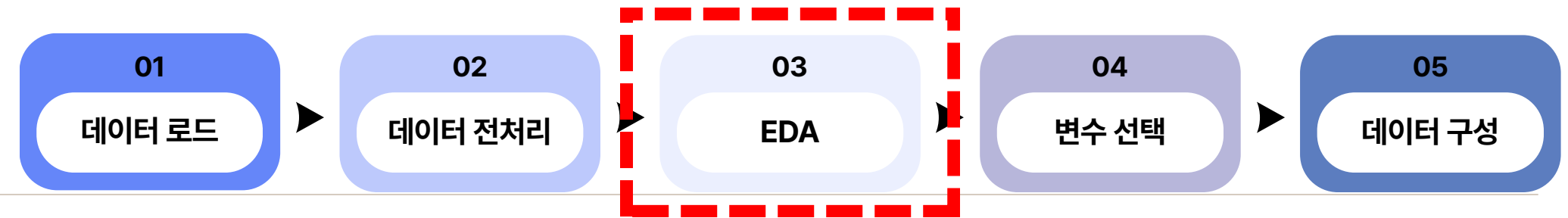
- battery_failure
- 0 : 정상
- 1 : 불량

• 분석 결과

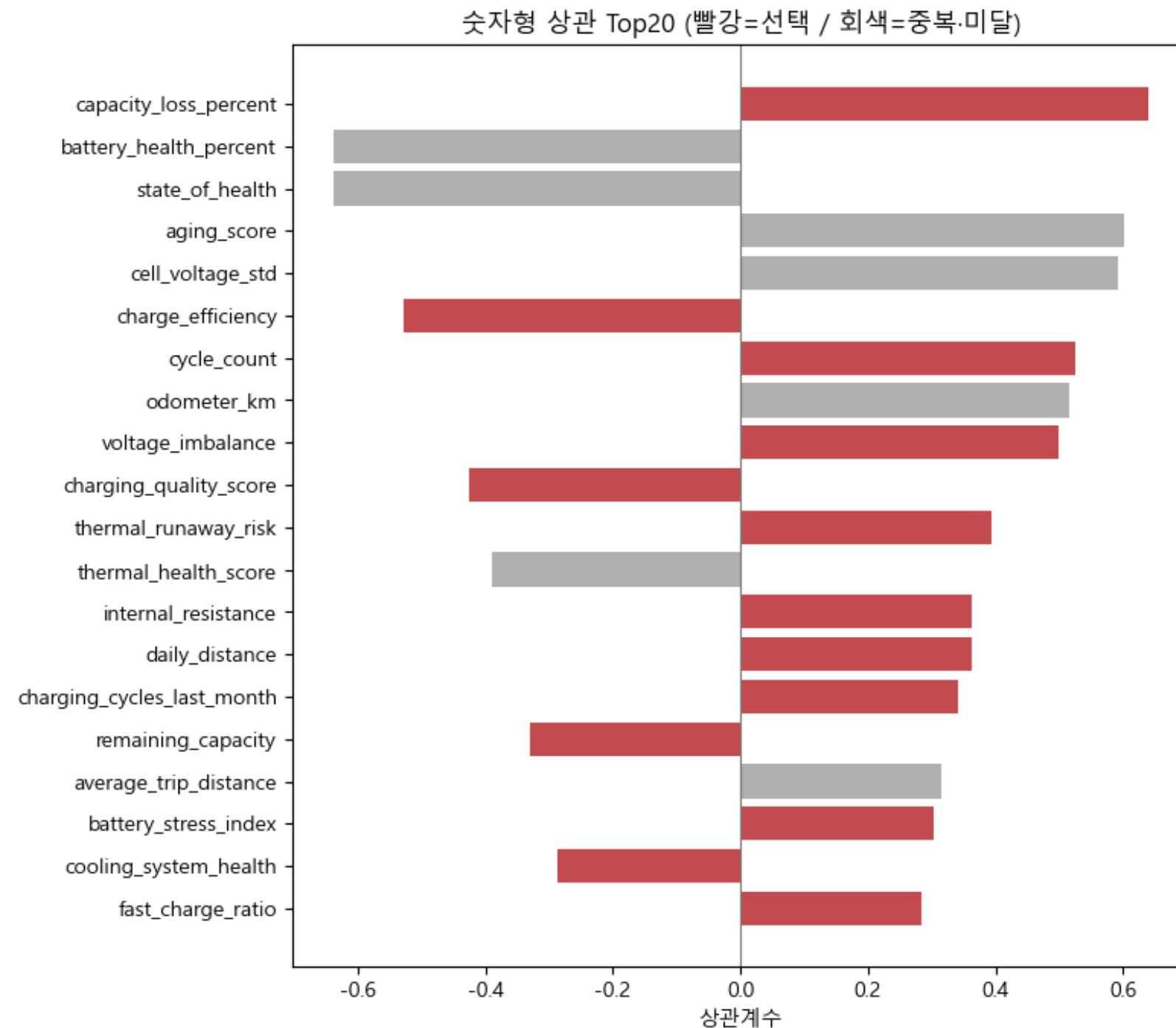
- 정상 데이터가 **약 90%**, 불량 데이터가 **약 10%**
- 정상 데이터가 불량 데이터보다 **약 9배 많은 클래스 불균형 확인**
- Accuracy만으로 평가할 경우 불량 탐지 성능을 충분히 반영하기 어려움

→ Recall·FN 중심 평가 + 불균형 처리 방법 비교 필요

데이터 수집 및 분석



* EDA ③ 타겟과 수치형 변수 상관관계 분석



• 분석 기준

- battery_failure와의 절대 상관계수
- $|\text{corr}| \geq 0.25$

• 분석 결과

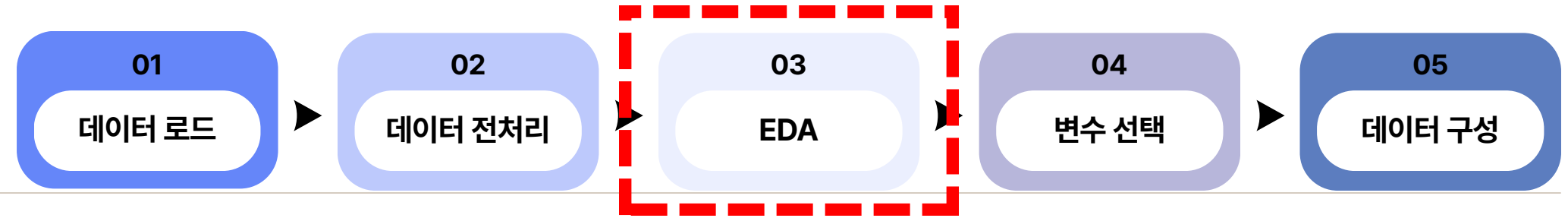
- 식별자 제외 후 수치형 변수 59개
- battery_failure와의 절대 상관계수 $|\text{corr}| \geq 0.25$ 적용
- 기준 충족 변수 25개 선정

• 인사이트

- 59개 변수 중 모든 변수가 불량 예측에 동일하게 유효하지 않음

→ 타겟 관련성이 확인된 25개 변수만 1차 후보로 선정

데이터 수집 및 분석



✳ EDA ④ 정상·불량 그룹별 변수 분포 비교

• 분석 기준

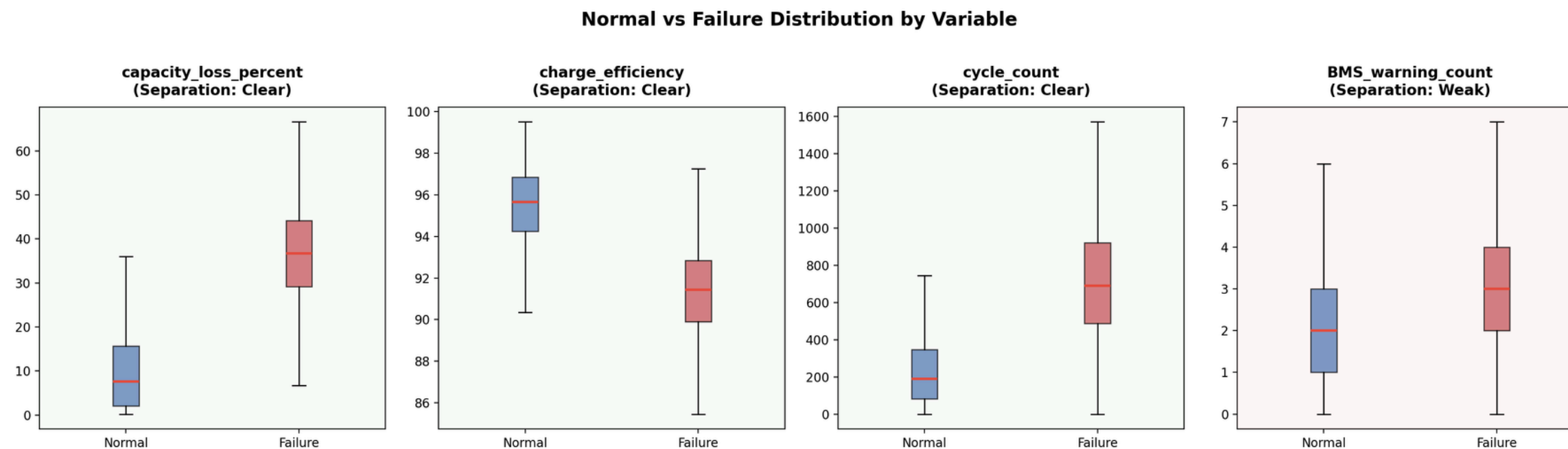
- 상관계수로 선택된 변수들을 정상(0)/불량(1) 그룹으로 나누어 박스플롯 비교
- 두 그룹의 상자 위치가 어긋날수록 해당 변수의 분리력이 높음

• 분석 결과

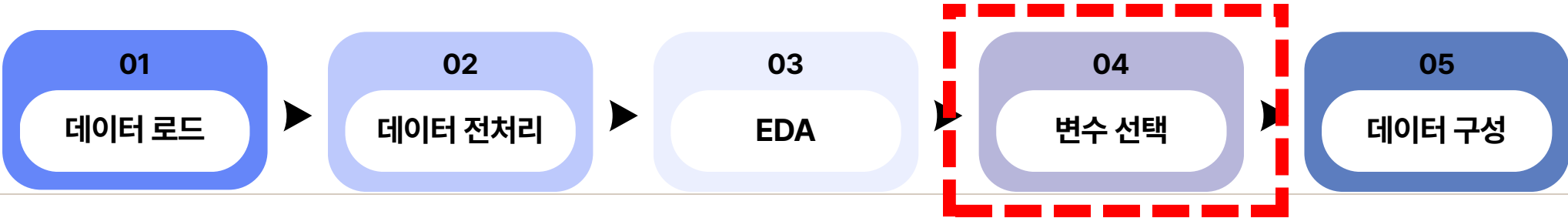
- capacity_loss_percent, charge_efficiency, cycle_count 등 → 정상/불량 상자가 뚜렷하게 분리됨
- BMS_warning_count → 두 그룹의 상자가 거의 겹침 (분리력 약함)

• 인사이트

- 정상·불량 간 분포 차이가 존재하는 변수를 확인 → **타겟과의 정량적 관련성을 상관분석으로 추가 검증**



데이터 수집 및 분석



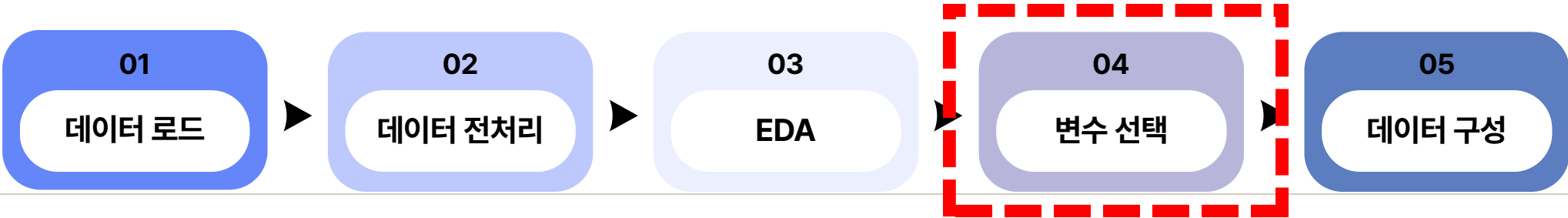
✳ EDA 기반 변수 선택 기준 수립

→ EDA에서 확인한 타깃 관련성과 변수 중복성을 기준으로 4가지 검증 방법을 적용

검증 방법	무엇을 확인?	적용 기준
Correlation	타깃과 선형 관계	$ corr \geq 0.25$
Mutual Information	타깃과 비선형 관계	상관분석 결과와 교차 검증
VIF	변수 간 다중공선성	$VIF < 10$ 확인
Cramér's V	범주형 변수와 타깃 간 연관성 확인	$V \geq 0.10$

단일 기준이 아닌 선형·비선형·중복성·범주형 연관성을 종합하여 변수 선택

데이터 수집 및 분석

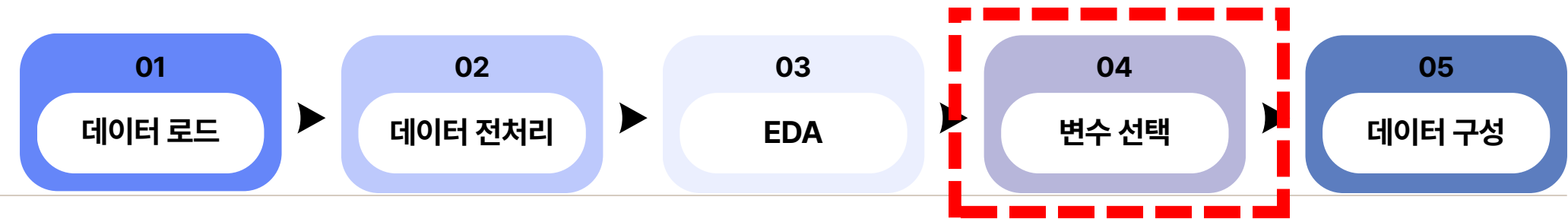


✳ EDA 기반 변수 선택 기준 수립

- Correlation — 타깃과 선형 관계 ($|corr| \geq 0.25$)
 - 숫자형 59개 전체 대상 → 25개 통과
 - 25개에서 변수 간 $|r| \geq 0.85$ 인 중복 쌍을 제거 → 16개
 - 제거된 9개 (중복 변수):

제거 변수	중복 대상 (잔존)
battery_health_percent	↔ capacity_loss_percent
state_of_health	↔ capacity_loss_percent
aging_score	↔ capacity_loss_percent
cell_voltage_std	↔ voltage_imbalance
odometer_km	↔ cycle_count
thermal_health_score	↔ thermal_runaway_risk
average_trip_distance	↔ daily_distance
sensor_fault_count	상대적 저상관
state_of_charge	상대적 저상관

데이터 수집 및 분석



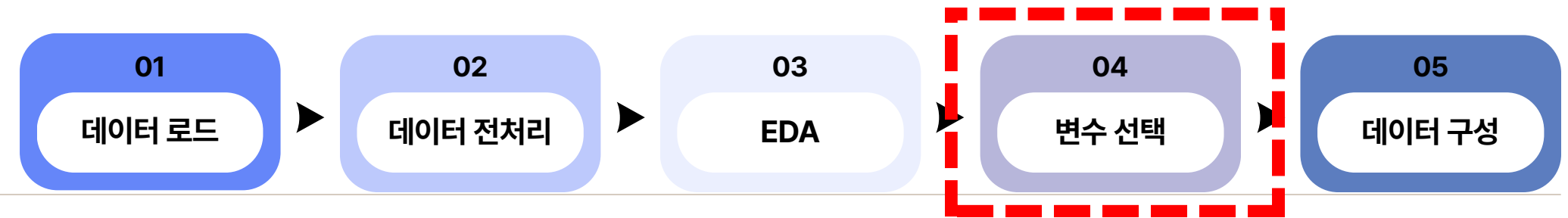
✳ EDA 기반 변수 선택 기준 수립

• Mutual Information — 비선형 관계 (교차 검증)

- 숫자형 59개 전체 대상으로 MI를 계산해 상관분석 결과와 교차 검증:

변수	MI	corr	상태
battery_health_percent	0.180	0.639	중복 제거됨
capacity_loss_percent	0.180	0.640	✓ 선택
state_of_health	0.177	0.637	중복 제거됨
cell_voltage_std	0.171	0.593	중복 제거됨
aging_score	0.164	0.601	중복 제거됨
predicted_remaining_life_cycles	0.145	0.255	✓ → 이후 누수 제외
voltage_imbalance	0.141	0.500	✓ 선택
charge_efficiency	0.128	0.527	✓ 선택
cycle_count	0.118	0.525	✓ 선택
odometer_km	0.116	0.515	중복 제거됨
thermal_runaway_risk	0.114	0.394	✓ 선택
internal_resistance	0.105	0.363	✓ 선택
thermal_health_score	0.102	0.390	중복 제거됨
charging_quality_score	0.099	0.425	✓ 선택
daily_distance	0.064	0.362	✓ 선택

데이터 수집 및 분석

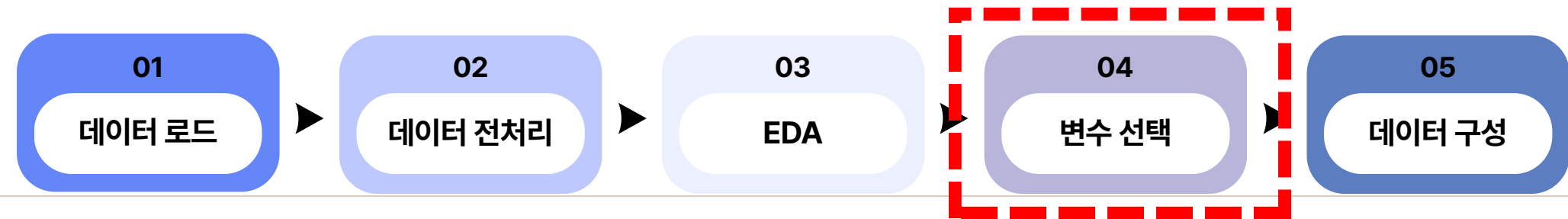


✳ EDA 기반 변수 선택 기준 수립

- VIF — 다중공선성 정밀 검증 ($VIF < 10$)
 - 필터링 후 남은 16개 대상으로 VIF 계산:

변수	VIF	판정
capacity_loss_percent	9.77	✓ < 10
cycle_count	6.04	✓ < 10
thermal_runaway_risk	4.42	✓ < 10
vehicle_age_years	4.26	✓ < 10
daily_distance	4.06	✓ < 10
cooling_system_health	3.82	✓ < 10
charging_cycles_last_month	3.34	✓ < 10
charging_quality_score	3.31	✓ < 10
charge_efficiency	3.14	✓ < 10
fast_charge_ratio	2.68	✓ < 10
predicted_remaining_life_cycles	2.53	✓ < 10
voltage_imbalance	2.35	✓ < 10
remaining_capacity	2.05	✓ < 10
internal_resistance	1.81	✓ < 10
battery_stress_index	1.58	✓ < 10
BMS_warning_count	1.23	✓ < 10

데이터 수집 및 분석

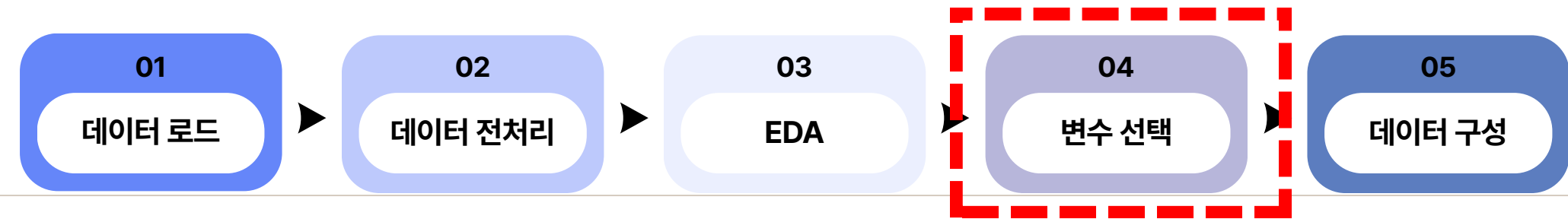


✳ EDA 기반 변수 선택 기준 수립

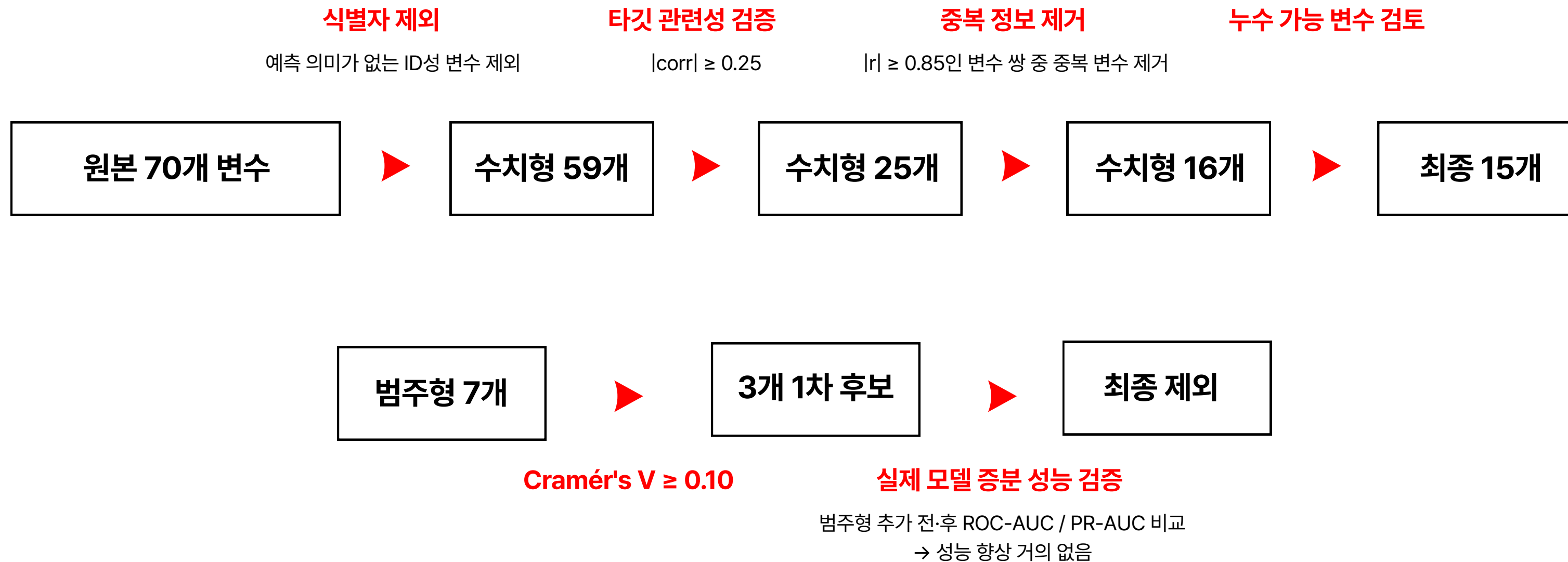
- Cramér's V — 범주형 변수와 타깃 간 연관성 ($V \geq 0.10$)
 - 범주형 7개 전체 대상:

변수	V	판정
fleet_or_private	0.342	✓ 통과
battery_chemistry	0.198	✓ 통과
vehicle_brand	0.133	✓ 통과
vehicle_type	0.096	x 미달
battery_manufacturer	0.006	x 미달
terrain_type	0.003	x 미달
drive_type	0.002	x 미달

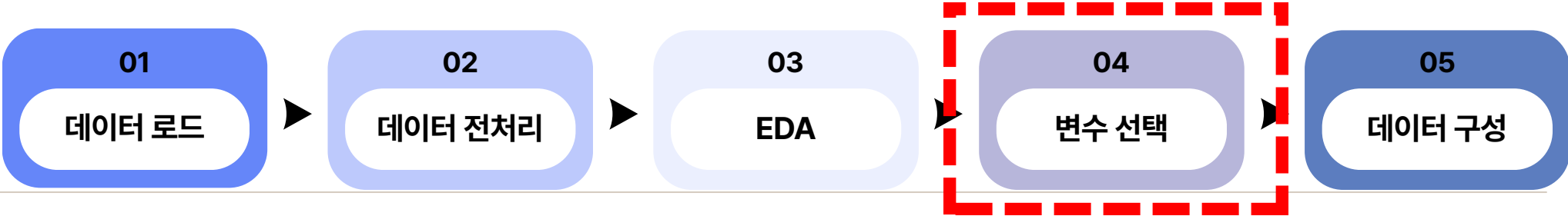
데이터 수집 및 분석



✳ 변수 선택 - 단계별 필터링



데이터 수집 및 분석



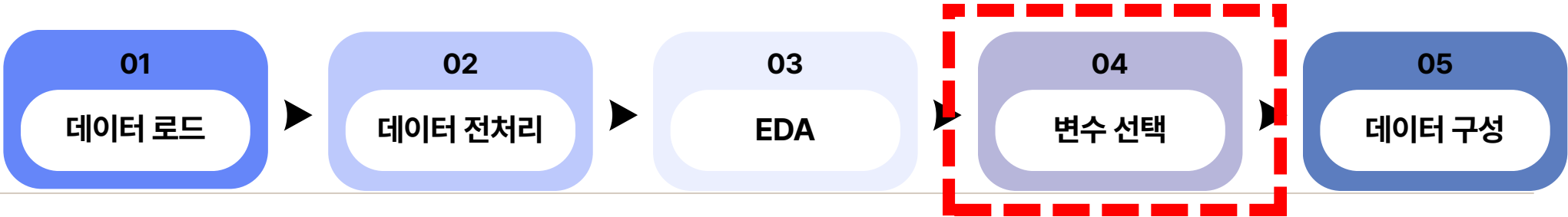
✱ 범주형 변수의 추가 예측 기여 검증

>>> 동일 모델·동일 검증 데이터 기준으로 범주형 변수 추가 전후 비교 (XGBoost 기준)

구성	ROC-AUC	PR-AUC
chemistry + fleet	0.9862	0.9026
chemistry + fleet + brand	0.9862	0.9029
숫자형만	0.9861	0.9025
숫자형 + fleet + chemistry	0.9862	0.9026

통계적 연관성은 확인되었으나 실제 예측 성능 개선은 미미
vehicle_brand, battery_chemistry, fleet_or_private → 최종 모델 입력에서 제외

데이터 수집 및 분석



✳ 최종 입력 변수 선정

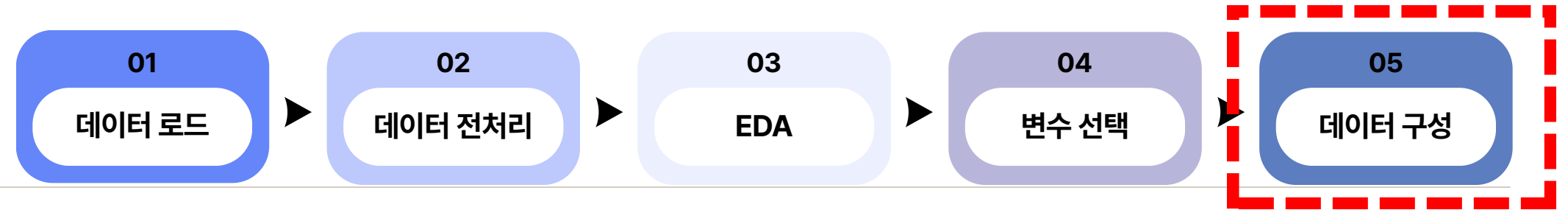
✳ 타깃 관련성 + 중복성 + 비선형 관계 + 실제 모델 기여도 종합 검증

70개 → 15개

최종 입력 변수는 모두 수치형

카테고리	변수명	설명
배터리 상태	capacity_loss_percent	용량 손실률 (%)
	remaining_capacity	잔존 용량 (Ah)
	internal_resistance	내부 저항 (Ω)
	voltage_imbalance	셀 전압 불균형
사용 패턴	cycle_count	충방전 횟수
	charging_cycles_last_month	최근 1개월 충전 횟수
	daily_distance	일일 주행거리 (km)
	fast_charge_ratio	급속 충전 비율
충전·열 품질	charge_efficiency	충전 효율 (%)
	charging_quality_score	충전 품질 점수
	thermal_runaway_risk	열폭주 위험도
	cooling_system_health	냉각 시스템 건강도
종합 지표	battery_stress_index	배터리 스트레스 지수
	BMS_warning_count	BMS 경고 횟수
차량 정보	vehicle_age_years	차량 연식 (년)

데이터 수집 및 분석



* 모델링 데이터 구성

최종 분석 데이터

154,036행



최종 입력 변수 선정

15개 변수



데이터 분리



입력 데이터 X

15개 변수

타깃 데이터 y

battery_failure

0 정상 / 1 불량

① 입력 데이터(X) 구성

- 배터리 불량 예측에 사용할 **최종 선정 변수 15개**
- 차량 및 배터리 상태 정보를 모델 입력값으로 구성

② 타깃 데이터(y) 구성

- Target: **battery_failure**
- 0 : 정상
- 1 : 불량
- 이진 분류(Binary Classification) 문제로 구성

③ 모델링 데이터 준비

- 입력 변수와 타깃 변수를 분리
- 이후 학습/검증/테스트 데이터 분할에 활용
- LightGBM 기반 불량 예측 모델 학습 준비

→ 이후 **Train / Validation / Test**로 분할하여 모델 학습

모델링 및 평가

✳ 모델링 설계

설계 항목	적용 내용	설계 이유
후보 모델	LightGBM / XGBoost	트리 기반 부스팅 모델을 후보로 설정하고 성능·학습시간 비교
데이터 분할	Train 60% / Val 20% / Test 20%, Stratify	클래스 비율을 유지하면서 학습·튜닝·최종 평가 데이터 분리
결측치 처리	Train 중앙값 기준 Imputer 적용	Val/Test 정보 유입을 방지하고 예측 시 동일한 전처리 기준 유지
학습 제어	Val AUC 기준 Early Stopping, patience 50	과적합을 방지하고 불필요한 반복 학습 감소

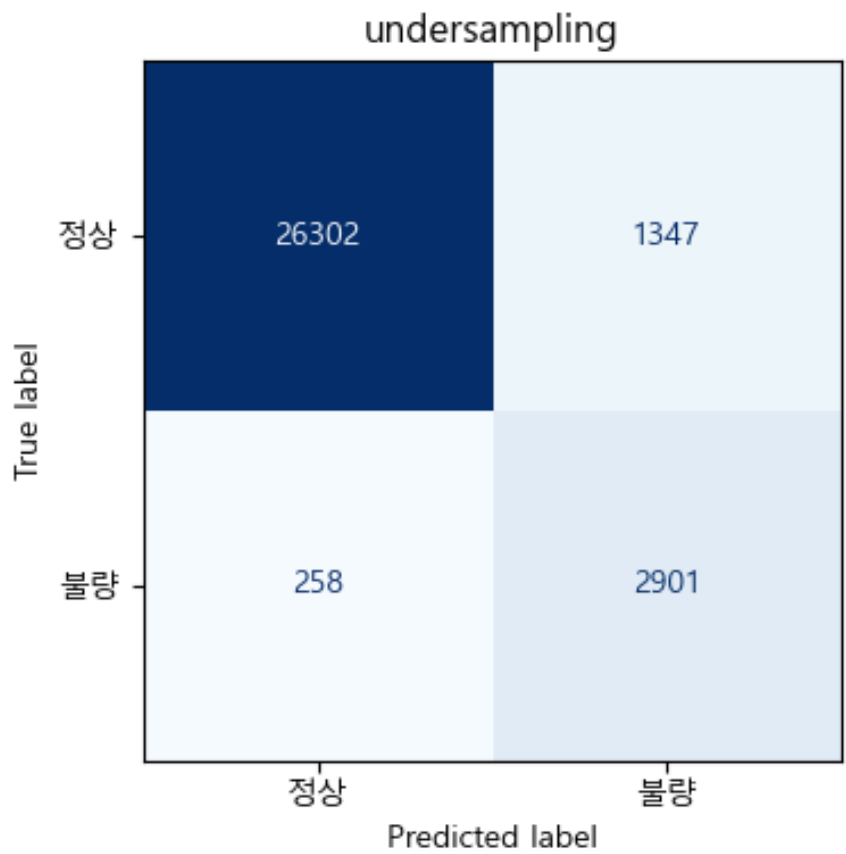
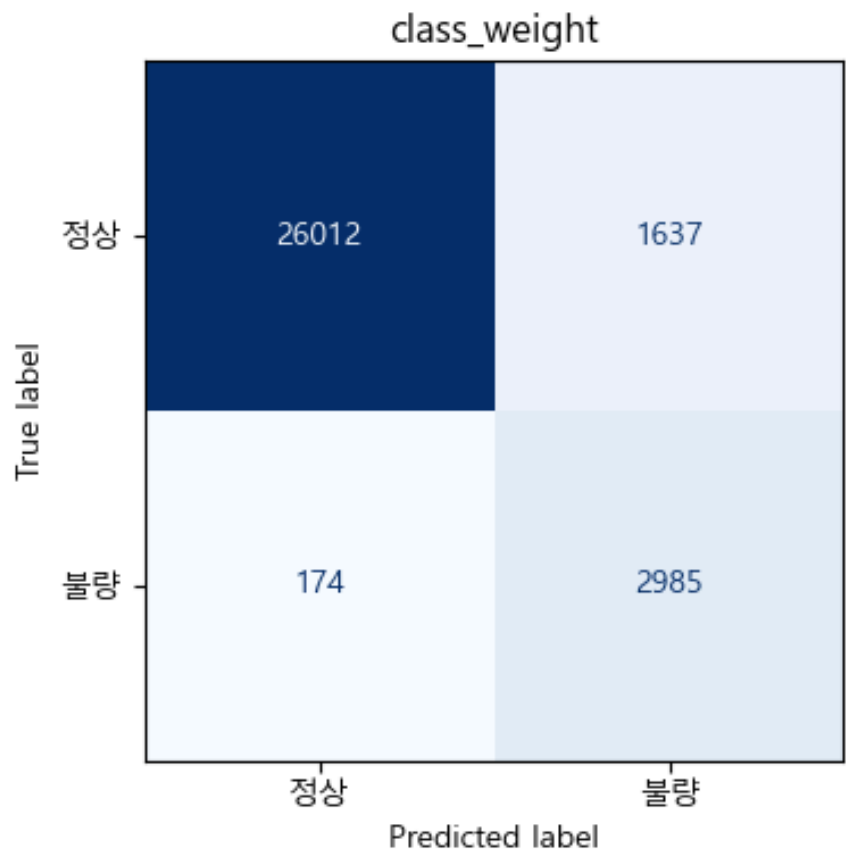


모델링 및 평가

✳ 불균형 처리 비교

방식	AUC	recall	precision	FN(농침)	FP(오탐)
class_weight='balanced'	0.9858	0.945	0.646	174	1,637
undersampling (2:1)	0.9853	0.918	0.683	258	1,347

- 배터리 불량은 '농침'이 치명적 → recall 우선 → class_weight 선택

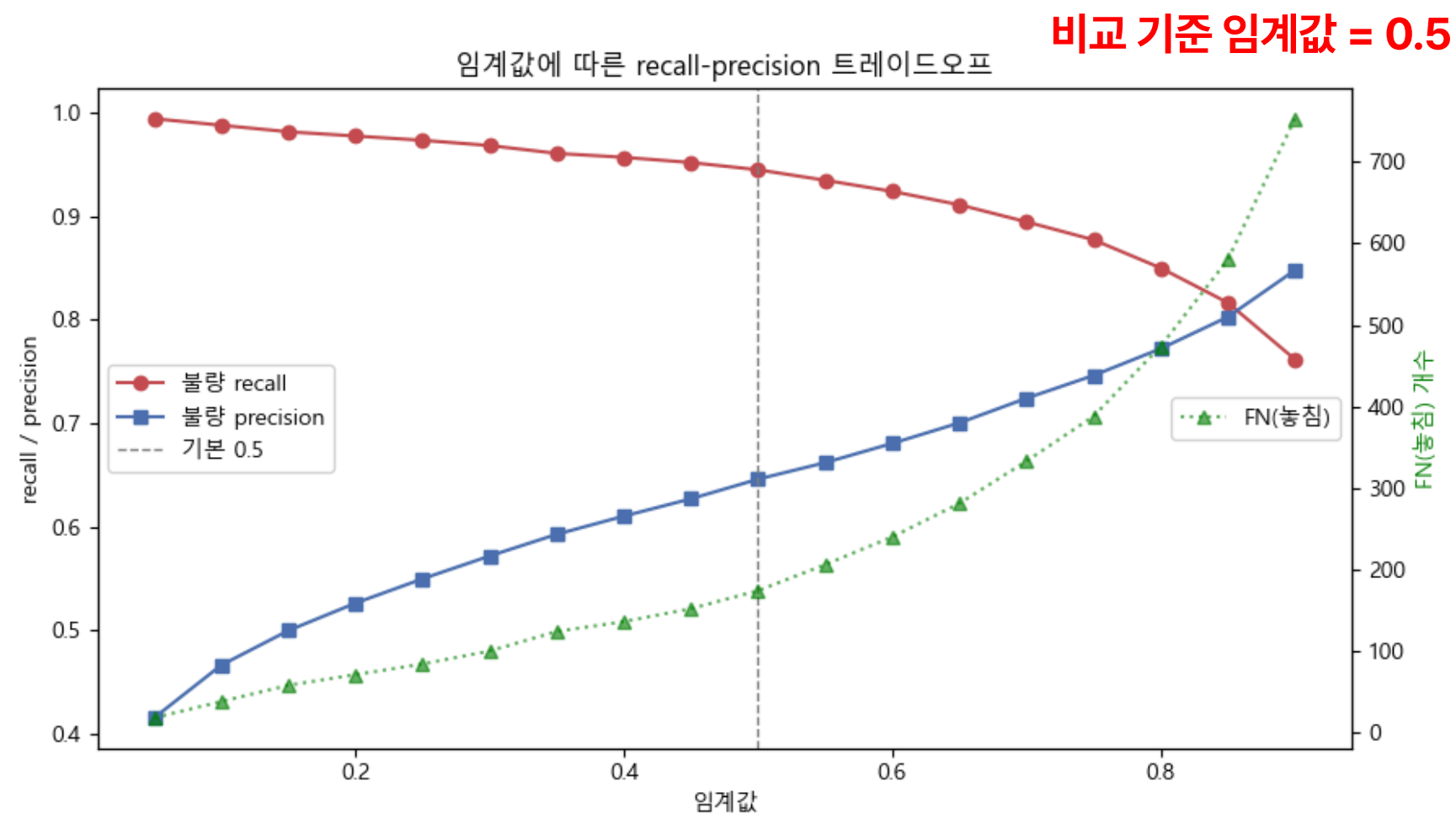


모델링 및 평가

✳ 임계값 조정 (0.5 → 0.2)

임계값	recall	FN(놓침)	FP(오탐)
0.5 (기본)	0.945	174	1,637
0.2 (선택)	0.978	71	2,783
0.1	0.988	38	3,576

- 놓침 174 → 71개로 절반 이하 감소
 - 오탐 증가는 추가 점검 비용, 놓침은 실제 고장 위험 → 허용 가능
 - 배터리 불량은 놓침(FN)의 비용이 높음
- Recall이 높은 class_weight='balanced' 선택



모델링 및 평가

✳ LightGBM vs XGBoost 비교

※ 동일 데이터 분할·입력 변수·평가 기준 적용

모델	AUC	recall	FN(농침)	학습시간
LightGBM	0.9858	0.978	71	3.5초
XGBoost	0.9853	0.943	181	11.3초

- LightGBM: 불량 110개 더 잡고, 3배 빠름

조합	AUC	recall	FN(농침)
LightGBM + class_weight	0.9858	0.978	71
LightGBM + undersampling	0.9853	0.973	87
XGBoost + class_weight	0.9853	0.943	181
XGBoost + undersampling	0.9856	0.973	85

- AUC는 유사하지만 Recall·FN·학습시간에서 LightGBM 우세

모델링 및 평가

✱ 최종 모델 요약

항목	내용
모델	LightGBM (class_weight='balanced')
입력 변수	15개
임계값	0.2 ※ Validation 데이터에서 선정
Test AUC	0.9858
Test recall	0.978 (농침 71개)
저장	lgbm_battery_model.pkl
핵심 변수	capacity_loss_percent



Test AUC
0.9858



Test Recall
0.978



FN (농침)
71건

 최종 모델: LightGBM + class_weight='balanced' + Threshold 0.2

정적 크롤링 - 전기차 충전소 정보 수집

✱ 위치 기반 충전소 검색 및 충전 방식별 추천 Engine

● requests 기반 충전소 정보 수집

- 사용자 입력 위치를 검색 파라미터로 변환하여 ChargeCheck 검색 페이지에 HTTP 요청
- 브라우저 자동화 없이 HTML을 직접 받아 빠르고 가볍게 충전소 정보 수집

● BeautifulSoup 기반 HTML 데이터 파싱

- HTML에서 충전소명, 주소, 전체·급속 이용 가능 충전기 수 추출
- 전체 이용 가능 수에서 급속 수를 제외하여 완속 이용 가능 충전기 수 계산

● 충전 방식별 추천 및 로컬 캐시 적용

- 완속·급속 이용 가능 충전기 수를 기준으로 필터링하고 내림차순 추천
- 동일 위치의 검색 결과를 10분간 CSV로 저장하여 반복 요청과 조회 시간 감소

동적크롤링 - EV 서비스센터 정보 수집

✳ 공식 제조사 센터 목록 동기화 및 초고속 로컬 검색 Engine

● 로컬 캐싱 기반 고속 검색 (Offline-First)

- 데이터를 한 번에 사전 수집해 로컬 DB로 활용함으로써 즉각적인 검색 속도 확보
- 실시간 요청 시 발생하는 대기 시간(3~10초) 및 서버 차단 위험 방지

● 제조사별 다변화된 데이터 수집

- 제조사별 보안·구조에 맞춘 최적화 수집 (Nissan: JSON 추출, VW: HTML 파싱, Tesla: 시드 데이터 백업)

● 지능형 주소 정규화 및 인접 지역 추천

- 서울특별시↔서울, 경기도↔경기 등 행정구역 표기를 통일하여 검색 정확도 향상
- 해당 지역 센터 부재 시 인근 시·도 센터를 자동으로 연결해 '검색 결과 0건' 방지

외부 API 연계 - 위치 기반 실시간 기온 조회

✳ 카카오 주소 검색 API와 기상청 단기실황 API 연계

● 카카오 API 기반 위치 좌표 조회

- 사용자가 입력한 주소를 카카오 주소 검색 API에 전달
- 검색 결과에서 x(경도), y(위도) 좌표 추출

● 기상청 격자 좌표 변환

- 카카오에서 받은 위도·경도를 기상청 전용 격자 좌표로 변환
- 변환 결과인 nx, ny와 조회 기준 날짜·시간 생성

● 현재 기온 조회 및 충전 방식 추천

- 기상청 단기실황 API 응답에서 T1H(현재 기온) 항목 추출
- 현재 기온이 30°C 이상이면 완속, 30°C 미만이면 급속 충전 추천

전기차 충전소 검색 Fallback 로직 개선 사항 issue

기존 방식(상세 주소 검색)의 핵심 한계점

- 즉시 검색 종료: 상세 주소 결과가 없으면 바로 오류 반환
- 검색 범위 제한: 상위 행정구역의 충전소를 추가 탐색하지 않음
- 사용자 불편: 직접 주소를 축약하여 재검색해야 함

검색 프로세스 Before vs. After

BEFORE (단일 주소 검색)

사용자 상세 주소 입력 → ChargeCheck 검색 → 결과 확인 → 결과 없음 시 즉시 오류 반환

AFTER (단계적 Fallback 검색)

상세 주소 입력 → 상세 지역 조회 → 결과 없음 → 상위 지역 재검색 → 충전소 발견 시 반환 및 캐시 저장

ex) 영통동 → 영통구 → 수원시 → 경기도

EV 서비스센터 크롤링 개선사항 및 구조 변경 사항 issue

기존 방식(카카오맵 연동)의 핵심 한계점

- 비공식 정비소 노출: 공식 서비스센터가 아닌 일반 사설 정비소가 검색 결과에 혼재되어 신뢰도 저하
- 탐색 시간 증가: 특정 시·군·구에 센터가 없을 경우, 상위 행정구역(시·도)으로 재검색해야 하는 시간 지연 발생
- 외부 의존성 및 제어 불가: 카카오맵 검색 알고리즘에 종속되어 브랜드별 동일 포맷 정보 제공 불가능

검색 프로세스 Before vs. After

BEFORE (카카오맵 연동)

사용자 입력 → 카카오맵 검색어 생성 → 외부 브라우저 실행 → 비공식 포함 결과 노출

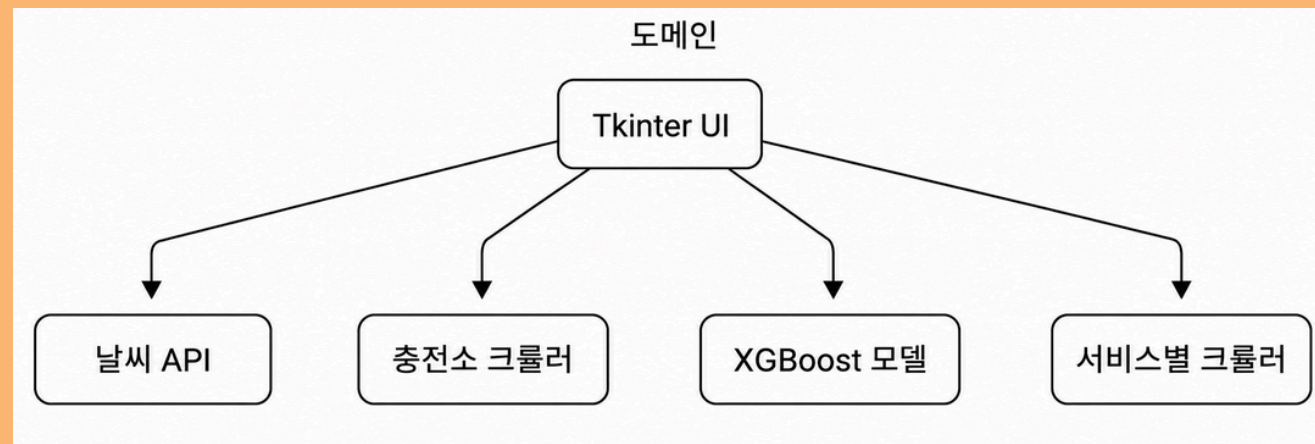
AFTER (로컬 공식 DB 기반)

브랜드 별 공식 사이트 일괄 수집 → 로컬 CSV 저장 → 사용자 입력 → 주소 정규화 & 로컬 조회 → 공식 센터 노출

시스템 설계 API 구조 [UI와 비즈니스 로직의 결합 문제를 해결하기 위해 FastAPI 도입]

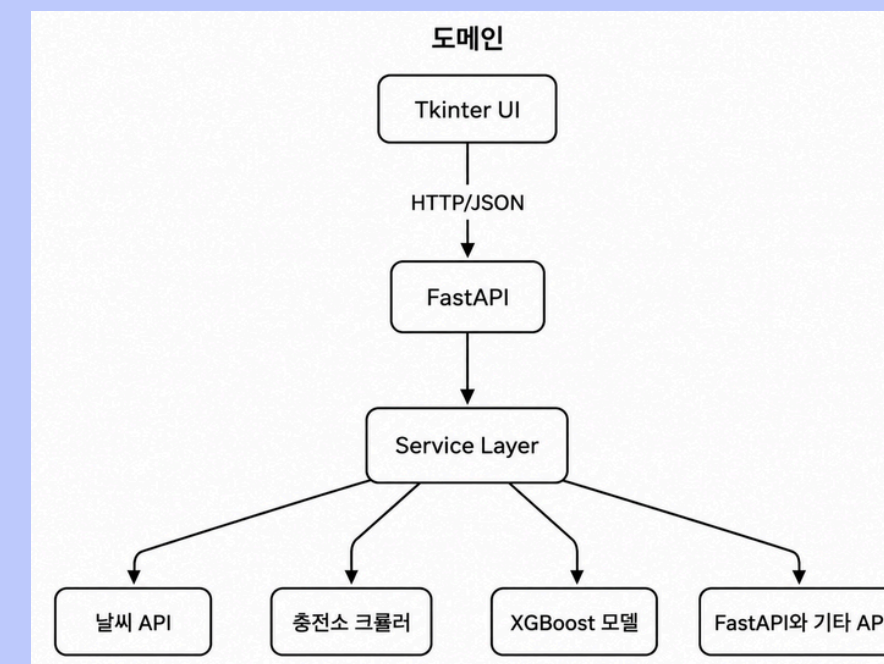
❄ 문제 해결 – FastAPI 기반 모델 구조 도입

UI와 핵심 기능의 강한 결합



- Tkinter가 모델, 크롤러, 외부 API를 직접 호출
- 기능 변경 시 UI 코드까지 함께 수정해야 하는 유지보수 문제
- 모델 기능을 다른 클라이언트에서 재사용하기 어려움

FastAPI 기반 API 계층 도입



- UI는 HTTP 요청과 결과 출력만 담당
- 백엔드는 기존 조회, 충전소 추천, 차량 진단 기능을 통합 처리
- Pydantic을 이용해 요청·응답 데이터 형식을 표준화

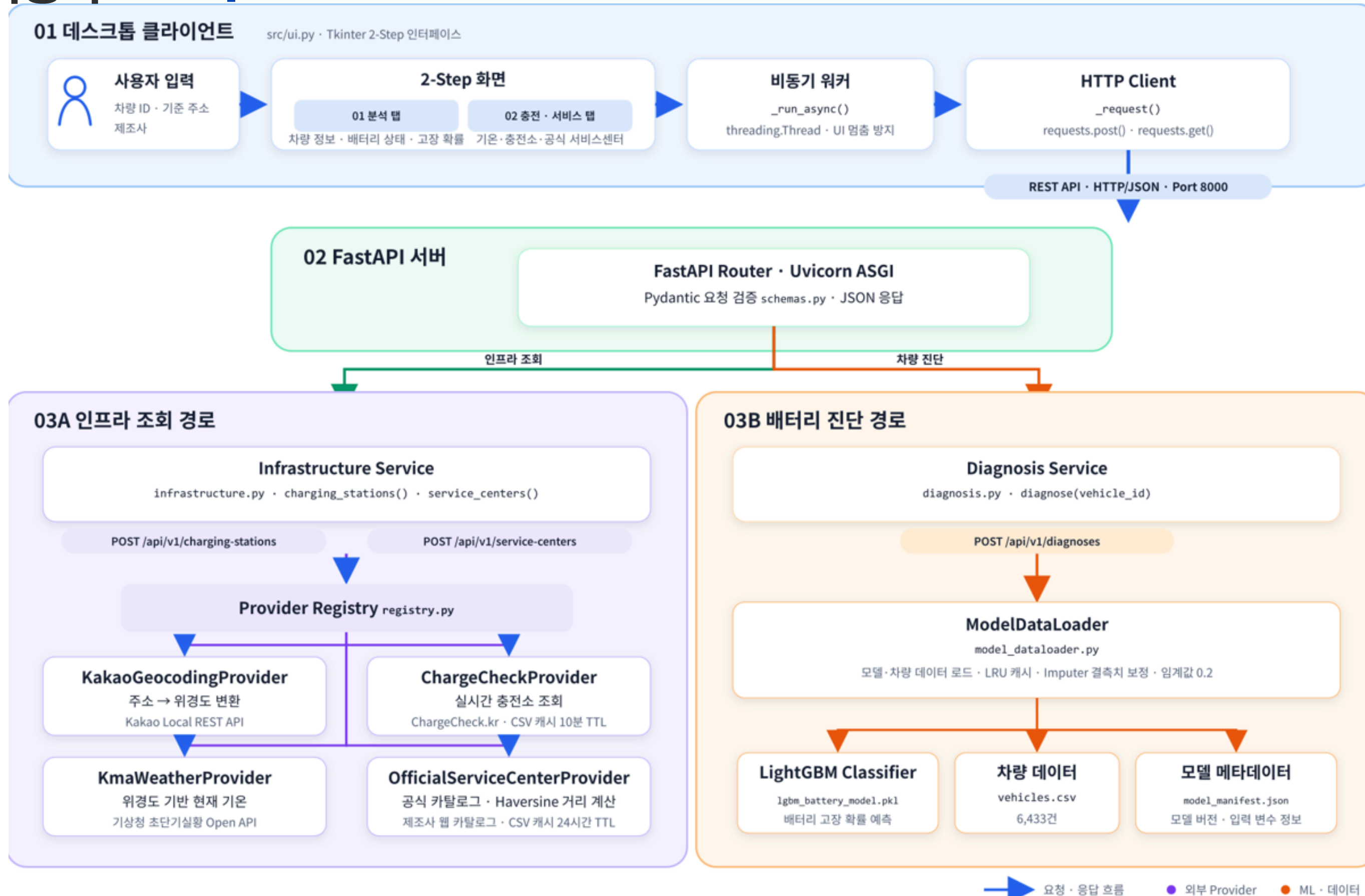
개선 결과

독립성과 확장성 확보

- Tkinter와 백엔드를 각각 독립적으로 테스트 가능
- Swagger를 이용한 API 단위 테스트 지원
- 향후 웹·모바일 UI에서도 동일한 백엔드 재사용 가능

시스템 최종 구조

[UI와 비즈니스 로직 결합을 위한 아키텍처]



시스템 구현 내용

[UI 작동 방법 및 결과]

EV Guard | 배터리 진단 · 인프라 안내

EV GUARD

01 분석

02 충전/서비스

REFERENCE DIAGNOSIS

BATTERY DIAGNOSIS

전기차 배터리 상태 분석

테스트 차량 ID로 모델 기반의 참고용 진단 결과를 확인합니다.

STEP 1 차량 조회 → 상태 분석 → 인프라 안내

01 차량 정보 입력

테스트 데이터에서 조회합니다.

차량 ID

EV282261

현재 주소 (선택)

서울 중구

데이터 추출 및 분석 시작 →

테스트 차량 ID 자동 입력

추출 데이터 미리보기

제조사	Nissan
차량 모델	Ariya
배터리	NMC
주행거리	30966.0
배터리 건강도	95.19
충방전 횟수	119.0
충전 품질 점수	100.0

02 분석 모델 개요

현재 활성 모델과 데이터 기준입니다.

모델 종류

모델 버전

입력 변수

확률 제공

LightGBM classifier (ver2)

lightgbm-ver2

15개

지원

충전 품질 점수

battery_stress_index

전압 불균형

열폭주 위험도

cooling_system_health

vehicle_age_years

BMS_warning_count

100%

88%

85%

79%

77%

65%

64%

모델 정보 로드 완료

배터리 건강도, 용량 손실, 노화-열 위험 지표를 중심으로 고장 위험을 분석합니다.

03 분석 결과

입력 차량의 예측 결과입니다.

배터리 상태

정상

Nissan · EV282261

고장 확률

0.0%

내 주변 인프라 안내 보기 →

합성 데이터 기반의 참고용 결과이며 실제 정비 확정 판정이 아닙니다.

시스템 구현 내용

[UI 작동 방법 및 결과]

EV Guard | 배터리 진단 · 인프라 안내

EV GUARD

01 분석02 충전/서비스

REFERENCE DIAGNOSIS

LOCAL INFRASTRUCTURE

가까운 충전소와 서비스센터

주소와 기온을 반영한 충전 방식, 제조사 공식 서비스센터를 함께 안내합니다.

STEP 2 주소 변환 → 기온 확인 → 맞춤 안내

01 검색 조건

주소와 제조사를 확인하세요.

기준 주소

서울 중구

제조사

Nissan

충전 방식

auto

주변 인프라 검색 →

현재 온도

28.6°C

급속 충전을 권장합니다.

주소 → Kakao 좌표 변환 → KMA 온도

조회 → 추천 충전소 검색

CHARGING STATION

추천 충전소

온도 및 선택 방식에 맞는 상위 결과입니다.

이름	주소	사용 가능	급속	완속
장교동 한화빌딩	서울 중구 청계천로 86	4	4	0
서울 서울역센트럴자이아파트	서울 중구 만리재로 175	26	2	24
서울 중구 실로암주차장	서울특별시 중구 중림로 49	2	2	0

OFFICIAL SERVICE CENTER

공식 서비스센터

제조사 공식 목록을 기준으로 검색합니다.

센터명	주소	거리	전화번호
닛산 성수 서비스	서울특별시 성동구 왕십리로 130	-	02-460-9999
닛산 일산 서비스	경기도 고양시 일산동구 백마로 522	-	031-810-1501
닛산 안양 서비스	경기도 안양시 만안구 만안로 100	-	031-443-7777
닛산 신갈 서비스	경기도 용인시 기흥구 용구대로 1845	-	031-810-1501

결론 및 향후 개선

[데이터셋 개요]

✳ 최종 요약 + 한계점 + 개선 방향

- 프로젝트 요약
 - 데이터 분석
 - 전처리 내용 : 분석에 혼돈을 주는 결측행 삭제 / 상관계수 및 다중공선 등을 고려한 학습 칼럼들 선정
 - 모델 선정 내용 : AUC Recall FN 지표를 토대로 XGB와 Undersampling을 비교 / 최종 LightGBM + class_weight 선정
 - 분석 성능 :
 - 크롤링
 - 충전소 : 사용자 주소 입력 > 전기차 충전소 Hub에서 충전소 위치 및 유형 크롤링
 - 충전 방식 제안 : 카카오 API로 위도 경도 크롤링 > 기상청 API로 기온 크롤링 > 기온 30°C 기준 완속 및 급속방식 제공
 - 서비스센터 : ID 기반 제조사 파악, 사용자 주소 입력 > 3사의 공식 홈페이지에서 서비스센터 위치를 크롤링
 - 서비스
 - 작동 방식 : 사용자는 본인의 차량 ID와 주소를 기입
 - 결과값 : 배터리의 상태 진단 / 주소지 기준 충전소와 서비스상태 위치
- 한계점 및 개선방향
 - 한계점
 - 학습 모델이 정상을 고장으로 분류하는 경우가 아직 많음.
 - auc가 0.98 이상으로 정확도가 지나치게 높아서 과적합의 가능성이 있음
 - 개선방향
 - 뷰/ 리액트를 사용해 기능별 분리를 더 깔끔하게 구현한다
 - 데이터를 csv가 아닌 데이터베이스로 확장해 성능과 안정성을 증가시킨다.

결론 및 향후 개선 [\[데이터셋 개요\]](#)

✱ 동영상 링크 첨부

경청해주셔서 감사합니다.

팀명 2조

팀원 서지우, 윤은빈, 지태현, 이태인, 홍인기, 김유석

